

Stultus

The search engine that thought it could kill god

*We made this 5 hours ago after an 11 hour code sprint. We're very tired. Please forgive us.

Building Block 3: The Weight System

We have three levels of weights working together:

1. **Field weights:** Title vs content vs keywords
2. **Tokenizer weights:** Word vs prefix vs n-gram (stored in `index_tokens`)
3. **Document weights:** Stored in `index_entries` (calculated: `field_weight × tokenizer_weight × ceil(sqrt(token_length))`)

Final Weight Calculation

When indexing, we calculate the final weight like this:

```
$finalWeight = $fieldWeight * $tokenizerWeight * ceil(sqrt($tokenLength));
```

When tokenizing, we use:

Why multiple tokenizers? Different strategies for different matches, another for partial matches, another for typos.

All tokenizers implement a simple interface:

```
interface TokenizerInterface
```

```
{
    public function tokenize(string $text): array; // Returns a
    public function getWeight(): int;             // Returns t
}
```

1 field-specific weights.

id_id	document_id	weight
-----	-----	-----
	42	2000
	42	500

becomes ["par", "pars", "parse", "parser"] (with min length autocomplete-like behavior.

es of a fixed length (I use 3). "parser" becomes ["par", "ars", "rse", "ser"] . This catches typos and partial word matches.

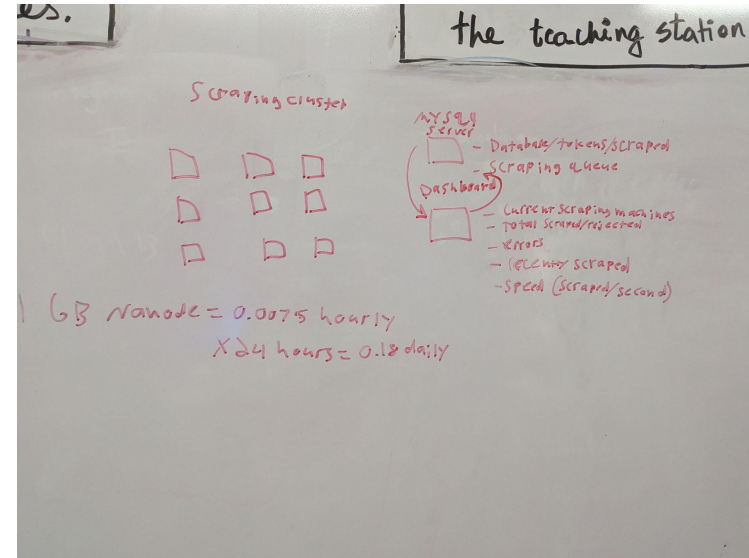
First, we extract words:

```
class NGramsTokenizer implements TokenizerInterface
{
    public function __construct(
        private int $ngramLength = 3,
        private int $weight = 1
    ) {}

    public function tokenize(string $text): array
    {
        $words = $this->extractWords($text);
```

The Plan - Initial

- Array of small machines running python web crawler, storing in a SQL database



“Is it really that easy?”

Jack - November 2025



No.



Not even close.

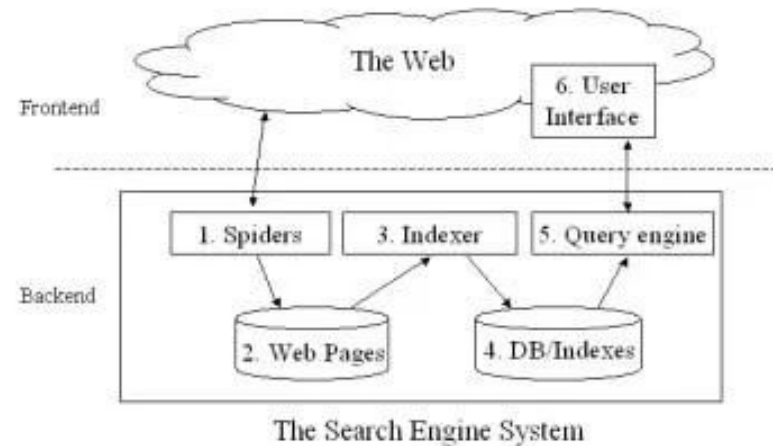




**How we took a simple
coding project way too far**

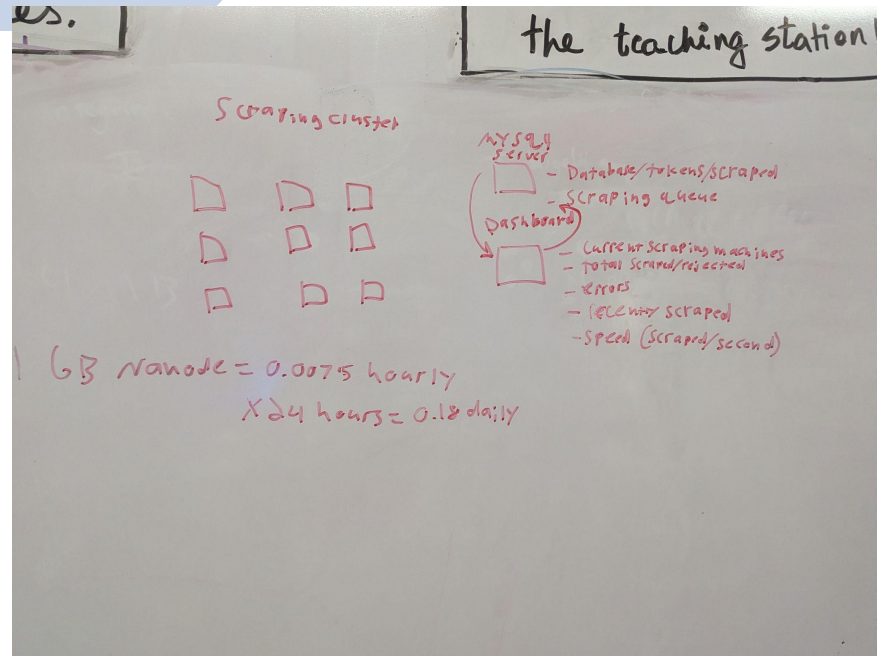
How Search Engines Work

- Crawlers visit web pages, store text in the index
 - Also add the urls in that page to a queue
- Data in the index is ranked somehow
- User searches a query
- Query is matched against index
- Returns best matches



Where it started

- Small cloud Linux VMs
 - 1 cpu
 - 1GB RAM
 - \$0.18 daily
- Sqlite3 cloud database
- Cloud dashboard/search



Uh Oh

#1 $\$0.18 * 30 = \5.4 per day

#1a I'm a broke college student

#2 Kubernetes is cool

#2a Kubernetes is expensive in the cloud

#2b see #1a

UH OH

Google's index is 400 billion documents?!

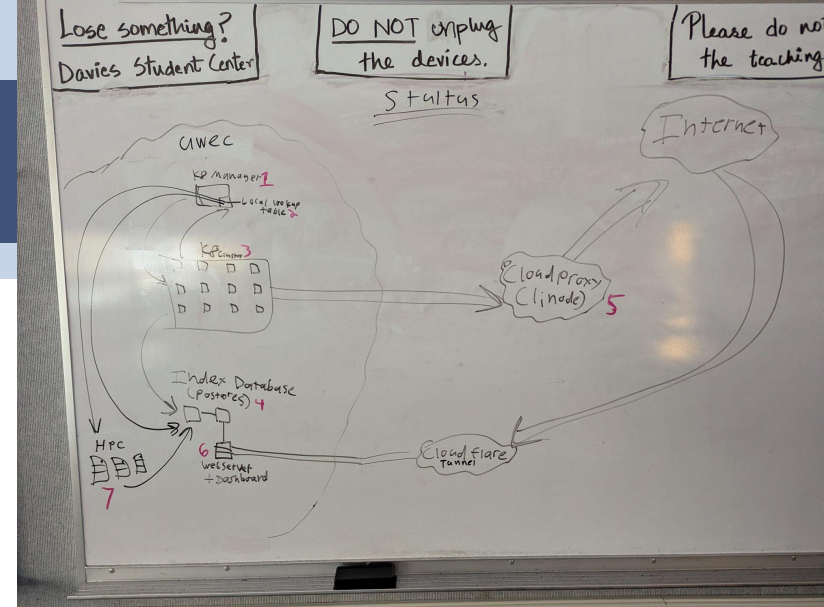
**Google's Index Size Revealed:
400 Billion Docs (& Changing)**

By [Cyrus Shepard](#) 

Updated: March 13, 2026

Where it ended up

- Local hardware
 - Wisconsin auction
- Kubernetes cluster running scrapers
- Postgres database
- Tunneling through cloud proxy

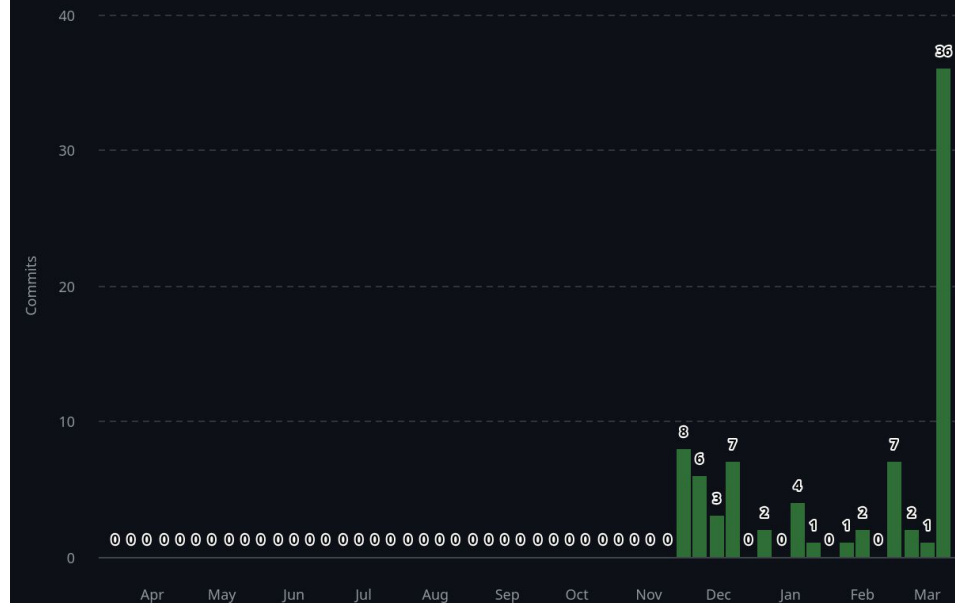


The Building

- Student Association for Computing Machinery
- Aaron, Silas, Reed, Jack
 - Also some other people
- Task planner
- *Lots* of groupchats
- No vibe coding

Commits

Number of commits per week



The Software

Crawlers

- Written in Python
- Attempts to filter out the unstructured web
 - Filter out # and ? (doesn't work)
- Beautiful debug statements
- Store urls in scraped pages in a url queue

```
# It iterates through the local_queue, scraping when the url is free to be scraped
# If a url is in the local_queue and cannot be scraped, it is added to queue_return_to_db, which
local_queue = []
queue_return_to_db = []
prev_base_domain = ""
page_not_in_english = False

while True:

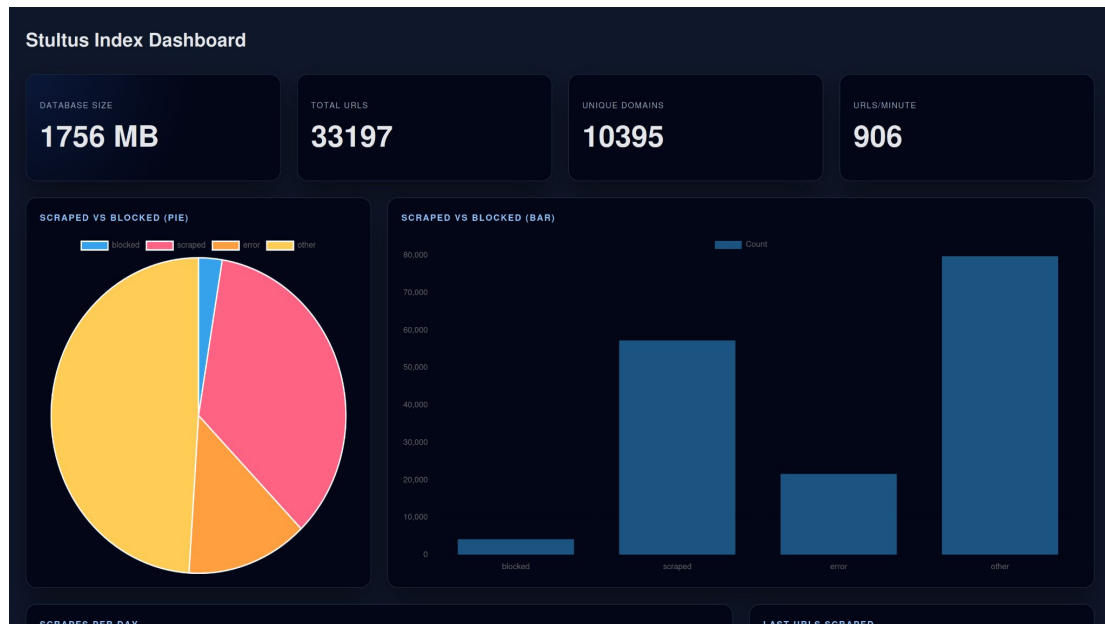
    # Iterates through the queue until it finds a domain which hasn't been scraped in the last 1
    # Holds a local queue of 30 urls so it doesn't need to interact with the db as much

    if len(local_queue) == 0:
        scraper.info_print("Reloading local queue")
        #print("Returning to queue:", queue_return_to_db)
        scraper.enqueue_urls(queue_return_to_db)
        #print("Queue to return", queue_return_to_db)
        #print("Enqueued")
        local_queue = scraper.get_next_urls(LOCAL_QUEUE_LENGTH)
        #print("Adding to lcal queue:")
        #print(local_queue)
        #print(local_queue)
```

Dashboard

What *stuff* are we tracking right now?

Dashboard



Dashboard

- Total URLs
- Unique Domains
- How many of our requests are blocked (a lot)
- Scrapers?
- Search speed?

Images - Use these as needed in your slides

https://drive.google.com/drive/folders/1YmV9gI0eFZWFO_Ltg3PM7rHVnRHX0yp8?usp=sharing

The Searching

The Algorithm at a High Level

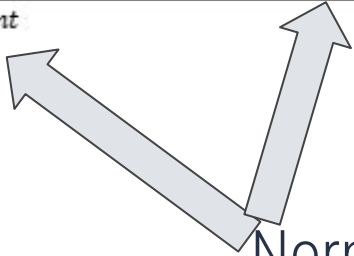
$$\text{searchOutput} = \text{relevance} * \text{references}$$

- Searching should include page relevance and popularity
- Relevance
 - Want pages to be relevant to searches
- References (aka popularity)
 - Prevents low quality, obscure web pages to rank high

The Algorithm - Relevance

$$\text{searchOutput} = \frac{\text{prefixScore} + \text{bigramScore} + \text{trigramScore} + \text{wordScore} + \text{wordCount}}{\text{wordCount}} * \text{references}$$

- prefixScore = $\text{ceil}(\text{sizeof}(\text{prefix})/3)$
- wordScore = wordMatches * 25
- wordCount = number of words on page
- bigramScore = number of bigrams on page
- trigramScore = number of trigrams on page



Normalization with respect to wordCount

The Algorithm - Bigrams and Trigrams

$$\text{searchOutput} = \frac{\text{prefixScore} + \text{bigramScore} + \text{trigramScore} + \text{wordScore} + \text{wordCount}}{\text{wordCount}} * \text{references}$$

- Tokenizes the page and stores the bigrams and trigrams into the DB
- Gets rid of duplicates in both search query and the DB
- Looks if the bigrams and trigrams in the search are in the page

```
words: ['disney']  
bigrams: ['ne', 'di', 'sn', 'ey', 'is']  
trigrams: ['dis', 'sne', 'ney', 'isn']  
prefixes: ['di']  
params: (['disney'], ['ne', 'di', 'sn', 'ey', 'is'], ['dis', 'sne', 'ney', 'isn'], ['di'])  
param count: 4
```

Search Output

```
(venv) PS C:\Users\User\OneDrive\Desktop\Stultus\search_engine> python search.py
```

```
Searching.... disney
```

```
words: ['disney']  
bigrams: ['ne', 'di', 'sn', 'ey', 'is']  
trigrams: ['dis', 'sne', 'ney', 'isn']  
prefixes: ['di']  
params: (['disney'], ['ne', 'di', 'sn', 'ey', 'is'], ['dis', 'sne', 'ney', 'isn'], ['di'])
```

```
Time taken: 0.2703850269317627
```

```
https://disneytermsofuse.com/ | score: 2.0006332623658603 | relevance: 1.4415584415584416 | ref_score: 1.387826677497038 | reference_count: 19
```

```
https://www.pinterest.com/disneystore/ | score: 1.947129191629646 | relevance: 1.5762711864406780 | ref_score: 1.23527550866827 | reference_count: 5
```

```
https://privacy.thewaltdisneycompany.com/en/ | score: 1.5880750475286682 | relevance: 1.1360000000000000 | ref_score: 1.397953386909039 | reference_count: 21
```

```
https://www.disneyadsales.com/our-brands/national-geographic/ | score: 1.538709227521145 | relevance: 1.1574074074074074 | ref_score: 1.3294447725782692 | reference_count: 11
```

```
https://www.youtube.com/user/wired/ | score: 1.509158150618792 | relevance: 1.1818181818181818 | ref_score: 1.2769799736005163 | reference_count: 7
```

```
https://allaboutcookies.org/ | score: 1.5 | relevance: 1.5000000000000000 | ref_score: 1.0 | reference_count: 1
```

```
https://www.reuters.com/ | score: 1.5 | relevance: 1.5000000000000000 | ref_score: 1.0 | reference_count: 1
```

```
https://www.youtube.com/MnSenateMedia | score: 1.4791226021355333 | relevance: 1.0909090909090909 | ref_score: 1.3558623852909055 | reference_count: 14
```

```
https://www.youtube.com/insidenatgeo/ | score: 1.4703895567255134 | relevance: 1.0909090909090909 | ref_score: 1.347857093665054 | reference_count: 13
```

```
https://skills.github.com | score: 1.4591460279304946 | relevance: 1.2500000000000000 | ref_score: 1.1673168223443957 | reference_count: 3
```

```
(venv) PS C:\Users\User\OneDrive\Desktop\Stultus\search_engine> █
```

The Algorithm - References

$$\text{searchOutput} = \text{relevance} * \left(2 - \frac{\sqrt[1.2]{\text{referenceCount}}}{\text{referenceCount}} \right)$$

- Need a normalized function that gives an output between 1 and 2
- Doesn't overpower relevance
- Doesn't grow too quickly or too slowly

$$2 - \frac{\sqrt[1.2]{1}}{1} = 1$$

$$2 - \frac{\sqrt[1.2]{10}}{10} = 1.318707931$$

$$2 - \frac{\sqrt[1.2]{100}}{100} = 1.535841117$$

$$2 - \frac{\sqrt[1.2]{1000}}{1000} = 1.683772234$$

$$2 - \frac{\sqrt[1.2]{10000}}{10000} = 1.784556531$$

Handwritten notes:

$$\text{ref_score} = 3 - \frac{\text{ref_cnt} + 1}{\text{ref_cnt}}$$

$\boxed{2 - \frac{\sqrt[1.2]{\text{ref_cnt}}}{\text{ref_cnt}}}$
 $\text{ref_cnt} = 1 \rightarrow 1$
 $\text{ref_cnt} = 10 \rightarrow 1.9$
 $\text{ref_cnt} = 100 \rightarrow 1.99$
 $\text{ref_cnt} = 1000 \rightarrow 1.999$

$$3 - \frac{1+1}{1} = 1$$

$$3 - \frac{10+1}{10} = 1.9 \text{ (B)}$$

$$3 - \frac{100+1}{100} = 1.99 \text{ (B)}$$

$$3 - \frac{1000+1}{1000} = 1.999 \text{ (B)}$$

$$3 - \frac{10000+1}{10000} = 1.9999 \text{ (B)}$$

Search Output

```
(venv) PS C:\Users\User\OneDrive\Desktop\Stultus\search_engine> python search.py
```

```
Searching.... disney
```

```
words: ['disney']  
bigrams: ['ne', 'di', 'sn', 'ey', 'is']  
trigrams: ['dis', 'sne', 'ney', 'isn']  
prefixes: ['di']  
params: (['disney'], ['ne', 'di', 'sn', 'ey', 'is'], ['dis', 'sne', 'ney', 'isn'], ['di'])
```

```
Time taken: 0.2703850269317627
```

```
https://disneytermsofuse.com/ | score: 2.0006332623658603 | relevance: 1.4415584415584416 | ref_score: 1.387826677497038 | reference_count: 19
```

```
https://www.pinterest.com/disneystore/ | score: 1.947129191629646 | relevance: 1.5762711864406780 | ref_score: 1.23527550866827 | reference_count: 5
```

```
https://privacy.thewaltdisneycompany.com/en/ | score: 1.5880750475286682 | relevance: 1.1360000000000000 | ref_score: 1.397953386909039 | reference_count: 21
```

```
https://www.disneyadsales.com/our-brands/national-geographic/ | score: 1.538709227521145 | relevance: 1.1574074074074074 | ref_score: 1.3294447725782692 | reference_count: 11
```

```
https://www.youtube.com/user/wired/ | score: 1.509158150618792 | relevance: 1.1818181818181818 | ref_score: 1.2769799736005163 | reference_count: 7
```

```
https://allaboutcookies.org/ | score: 1.5 | relevance: 1.5000000000000000 | ref_score: 1.0 | reference_count: 1
```

```
https://www.reuters.com/ | score: 1.5 | relevance: 1.5000000000000000 | ref_score: 1.0 | reference_count: 1
```

```
https://www.youtube.com/MnSenateMedia | score: 1.4791226021355333 | relevance: 1.0909090909090909 | ref_score: 1.3558623852909055 | reference_count: 14
```

```
https://www.youtube.com/insidenatgeo/ | score: 1.4703895567255134 | relevance: 1.0909090909090909 | ref_score: 1.347857093665054 | reference_count: 13
```

```
https://skills.github.com | score: 1.4591460279304946 | relevance: 1.2500000000000000 | ref_score: 1.1673168223443957 | reference_count: 3
```

```
(venv) PS C:\Users\User\OneDrive\Desktop\Stultus\search_engine> █
```

The Database

- Words
- Bigrams
- Trigrams
- Prefixes
- Urls
- Urls_references
- Entities
- Entity_urls
- Url_token_counts
- Url_queue
- Bigram_urls
- Trigram_urls
- Prefix_urls
- Word_urls
- Weights

The Hardware

The Hardware

- Used desktops from the Wisconsin auction
- University server room
- Old supercomputing rack + switch + PDUs



The Hardware (cont.)



Networking

Networking is the hardest.

The UWEC network is made to be secure and block potentially dangerous things...

Which is not conducive for crawling the web.



The Clustering

Kubernetes

What is Kubernetes

Kubernetes is a piece of software that clusters together physical computers.

Kubernetes

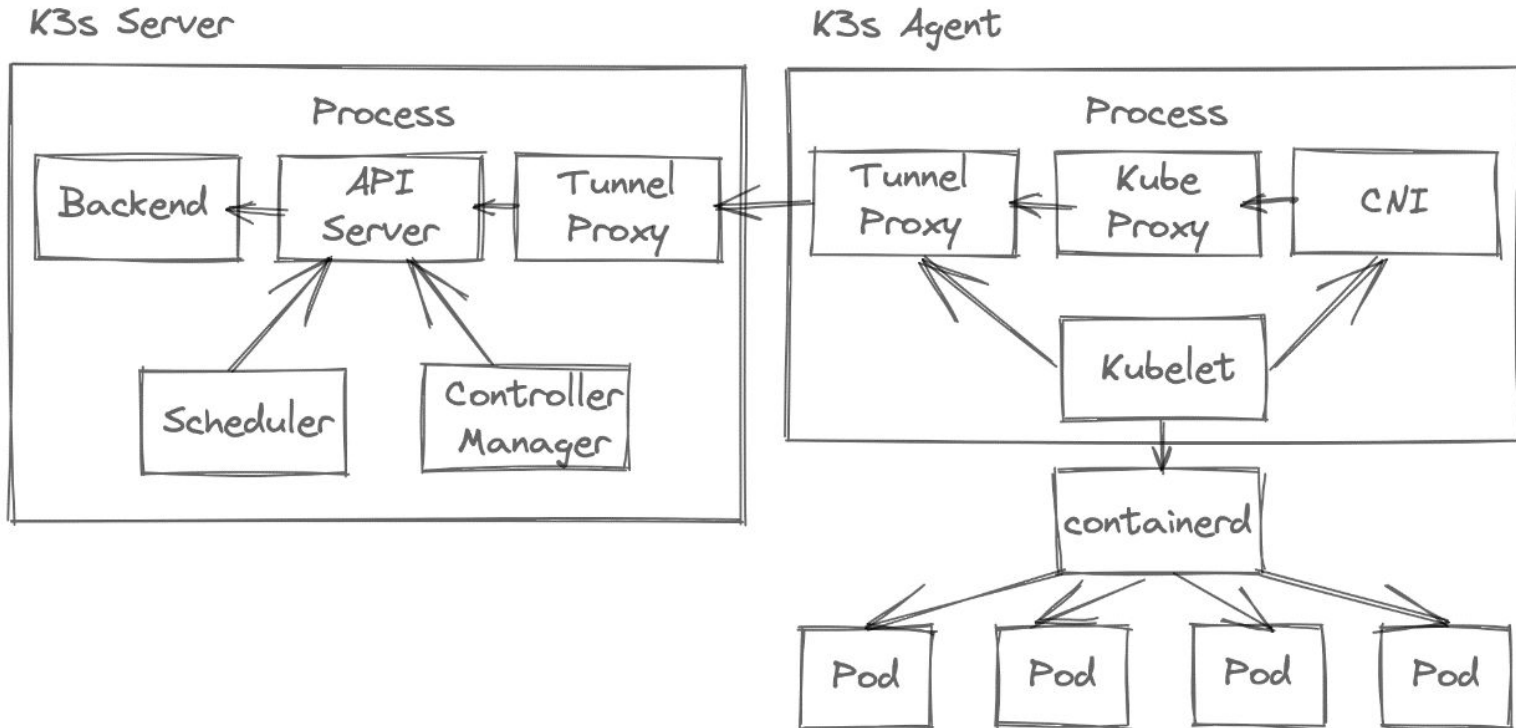
Kubernetes is hard. And Reed ran out of time to configure a cluster and deploy pods. :(

However, the plan is to eventually use K3S to cluster all the server.

Kubernetes would allow us to deploy crawling pods on the hardware in within the cluster.



Kubernetes (cont.)



Future Work

This is a big part

Searching

- Precompute some of the quantities for constant time lookup
- Dwell time scoring
- Explore a RAG solution

Filtering

- Filter out pictures and files
- Many URLs don't need to be crawled
- Crawls redirect links

None

<https://firebase.google.com>

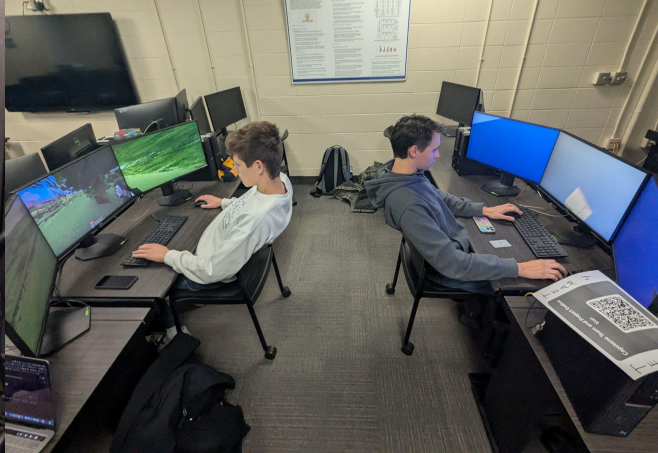
None

<https://developer.chrome.com/home>

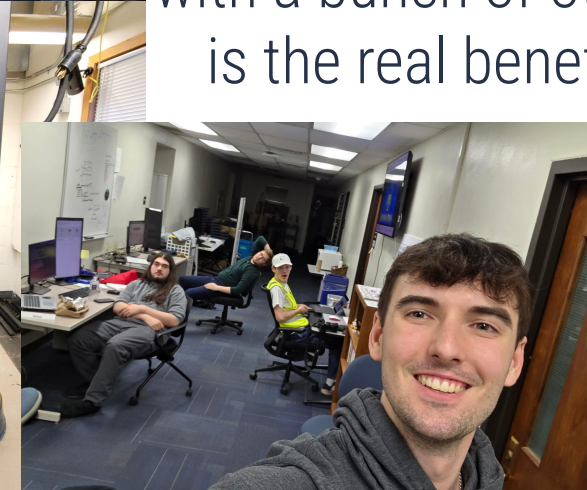
None

<http://www.snapchat.com/add/uweauclaire>





“Hanging out and building stuff
with a bunch of other smart people
is the real benefit of university”



Live Demo!!!



Click to add title



Now You Try!

<https://search.stultus.rip>

Disclaimer: we haven't tested it with more than a couple people so performance may not be great.

(It may explode so don't blame us if it breaks)